

JOBSHEET

PENGEMBANGAN MEDIA PEMBELAJARAN **INTERNET OF THINGS (IoT) UNIIOT BERBASIS JAVASCRIPT DENGAN **TEKNOLOGI WEBSOCKET** DAN **MYSQL** PROGRAM STUDI PENDIDIKAN TEKNIK MEKATRONIKA**

Oleh:

Adrian Ramdhany

Pembimbing:

Amelia Fauziah Husna, S.Pd., M.Pd.



KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas keberhasilan penyusunan dokumen *Jobsheet* Pengembangan Media Pembelajaran *Internet of Things* (IoT) UNIOT. Modul praktik ini beroperasi menggunakan bahasa pemrograman JavaScript dengan dukungan teknologi protokol *WebSocket* dan sistem basis data MySQL. Penulis menyusun instrumen pembelajaran ini untuk melengkapi luaran penelitian akademik pada Program Studi Pendidikan Teknik Mekatronika, Universitas Negeri Yogyakarta

Jobsheet ini memfasilitasi mahasiswa dalam membangun dan menguji arsitektur jaringan *Client-Server* secara mandiri. Mahasiswa akan menjalankan praktik komunikasi data secara langsung menggunakan perangkat fisik mikrokontroler ESP32-S3 dan platform perancangan aplikasi seluler MIT App Inventor. Penulis menstrukturkan tahapan praktikum secara sistematis agar pengguna mampu menganalisis konsep transmisi data dua arah secara seketika (*real time*) tanpa hambatan latensi.

Penyelesaian penyusunan modul ini terwujud berkat kontribusi akademik dari berbagai pihak. Penulis mengucapkan terima kasih secara khusus kepada Ibu Amelia Fauziah Husna, S.Pd., M.Pd. selaku dosen pembimbing yang terus memberikan evaluasi dan arahan sistematis selama proses pengembangan purwarupa. Penulis menyadari *platform* UNIOT dan dokumen penyertanya memerlukan pembaruan berkelanjutan. Penulis menerima kritik dan saran teknis yang konstruktif guna menyempurnakan ekosistem pembelajaran ini pada masa mendatang.

Yogyakarta, 23 April 2026

Adrian Ramdhany
NIM 22518241019

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
JOBSHEET 1.....	4
A. Tujuan Praktikum.....	4
B. Capaian Pembelajaran Mata Kuliah.....	4
C. Dasar Teori.....	4
D. Alat dan Bahan.....	6
E. Keselamatan Kerja	6
F. Langkah Kerja	6
G. Tugas	10
H. Dokumentasi	10
JOBSHEET 2.....	11
A. Tujuan Praktikum.....	11
B. Capaian Pembelajaran Mata Kuliah.....	11
C. Dasar Teori.....	12
D. Alat dan Bahan.....	12
E. Keselamatan Kerja	13
F. Langkah Kerja	13
G. Tugas	24
H. Dokumentasi	25
JOBSHEET 3.....	26
A. Tujuan Praktikum.....	26
B. Capaian Pembelajaran Mata Kuliah.....	27
C. Dasar Teori.....	27
D. Alat dan Bahan.....	29
E. Keselamatan Kerja	29
F. Langkah Kerja	29
G. Tugas	40
H. Dokumentasi	40
PENUTUP.....	41

JOBSHEET 1

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK		
	Semester:	LS 1: Pengenalan <i>Platform</i> UNIOT dan Komunikasi <i>Publish-Subscribe</i> untuk <i>Monitoring</i> dan Kendali Jarak Jauh Berbasis <i>WebSocket</i> UNIOT	4 x 50 Menit
	Revisi: 00	Tanggal:	

A. Tujuan Praktikum

1. Mahasiswa mampu menunjukkan sikap bertanggung jawab, mandiri dan disiplin dalam menyelesaikan setiap tahapan pekerjaan praktik di bidang mekatronika, khususnya yang berkaitan dengan penerapan sistem *Internet of Things*.
2. Mahasiswa mampu mengonfigurasi dan mempraktikan komunikasi IoT melalui berbagai *broker* dan *platform* (termasuk UNIOT), baik dalam mode *publisher* maupun *subscriber*, serta mampu mengintegrasikannya dengan aplikasi mobile berbasis Android.

B. Capaian Pembelajaran Mata Kuliah

1. Mahasiswa mampu memahami dan mengimplementasikan protokol yang populer digunakan dalam pengembangan sistem berbasis *Internet of Things*.
2. Mahasiswa mampu membuat, merancang dan mengembangkan sebuah sistem sederhana yang menerapkan konsep *Internet of Things*.

C. Dasar Teori

1. *Internet of Things*

Internet of Things (IoT) adalah konsep yang menghubungkan berbagai perangkat listrik ke internet. IoT diterapkan untuk mengembangkan sistem pengukuran berbasis mikrokontroler ESP32-S3

dan MySQL. Pemantauan dan pengelolaan data secara *real time* dapat dilakukan melalui aplikasi antarmuka.

2. Protokol Komunikasi *WebSocket*

WebSocket adalah protokol komunikasi yang memungkinkan komunikasi dua arah (*full-duplex*) secara *real time* antara *client* dan *server* melalui satu koneksi yang berkelanjutan. Berbeda dengan protokol HTTP tradisional yang merupakan komunikasi setengah-dupleks (data dikirim dan diterima bergantian), *WebSocket* memungkinkan data mengalir secara simultan di kedua arah, sehingga mengurangi latensi dan meningkatkan efisiensi jaringan.

Karakteristik ini sangat sesuai untuk media pembelajaran IoT, sebab *data sensor* perlu ditampilkan segera pada *dashboard*, sementara perintah pengguna juga harus diteruskan ke perangkat tanpa jeda yang panjang. *WebSocket* digunakan sebagai penghubung utama antara ESP32, *server*, dan *dashboard*. Perangkat IoT mengirim data sensor ke *server* melalui koneksi *WebSocket*. *Server* kemudian meneruskan data tersebut ke *dashboard* yang sedang aktif. Pada saat yang sama, *dashboard* juga dapat mengirim perintah balik ke perangkat melalui jalur yang sama. Pola ini membentuk komunikasi dua arah yang sederhana namun sangat kuat untuk kebutuhan pembelajaran.

3. *Publisher* dan *Subscriber*

Sistem mulai bekerja saat perangkat *client* mengirimkan *handshake request* ke Alamat *server*. Setelah koneksi *WebSocket* terbentuk, *server* membentuk *tunnelling* (terowongan) komunikasi. Jalur ini akan selalu dalam kondisi siap selama *client* terhubung dalam jaringan. Metode ini memastikan aliran data berjalan secara *instant* tanpa perlu melakukan inisialisasi ulang pada setiap pengiriman data.

a. *Publish*

Proses pengiriman data (*publish*) akan aktif saat *client* memiliki data untuk dikirim ke *server*. Data harus dikemas dalam bentuk JSON

(*JavaScript Object Notation*). Struktur JSON ini memuat dua parameter yaitu *variable* (*var*) sebagai identitas dan nilai (*val*) sebagai isi data.

b. *Subscribe*

Proses *subscribe* bekerja secara reaktif saat *client* berada dalam mode siaga (*listening*) untuk menerima data dari *server*. Saat *server* mengirim sebuah paket JSON baru, *client* langsung menangkap secara otomatis tanpa melakukan *request* ulang.

Pada proses ini, *client* melakukan proses ekstrasi (*parsing*) terhadap struktur JSON untuk mengidentifikasi perintah kendali atau membaca nilai yang masuk. Integrasi antara pengiriman dan penerimaan ini membentuk sebuah siklus komunikasi yang cepat dan stabil.

D. Alat dan Bahan

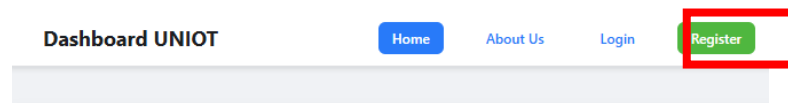
- PC atau Laptop yang terhubung ke Wi-Fi Laboratorium

E. Keselamatan Kerja

- Gunakan pakaian kerja atau jas laboratorium sesuai standar keselamatan saat melakukan kegiatan eksperimen.
- Pastikan area kerja atau meja praktikum selalu dalam keadaan kering, bersih, dan bebas dari paparan cairan.
- Jauhkan benda konduktif yang tidak diperlukan dari area kerja untuk mencegah interferensi elektromagnetik.

F. Langkah Kerja

1. Sambungkan laptop ke Wi-Fi yang ada pada laboratorium (tanyakan kepada Dosen atau Instruktur).
2. Akses alamat *platform* UNIOT pada *Browser* (contoh <http://192.168.1.50:3001>),
3. Jika sudah diakses, buat akun terlebih dahulu pada bagian *Register*.

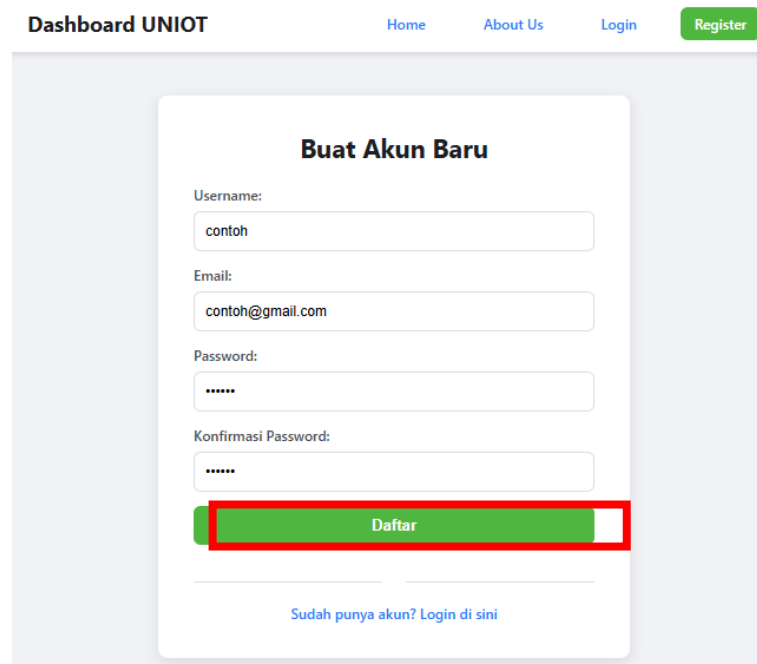


Selamat Datang di Platform UNIOT

Ini adalah halaman utama untuk proyek monitoring dan kontrol IoT Anda. Platform ini dirancang untuk memungkinkan pengguna mendaftarkan perangkat (seperti ESP32), mengirim data sensor, dan mengontrol aktuator secara "real-time" melalui dashboard yang dapat disesuaikan.

Untuk memulai, silakan [Login](#) jika Anda sudah memiliki akun. Jika Anda pengguna baru, Anda dapat [membuat akun baru](#) untuk mendaftarkan perangkat pertama Anda.

4. Isikan *username*, *email*, dan *password*. Lalu klik “Daftar”. Setelah berhasil otomatis akan membuka halaman login lalu lakukan *login* sesuai dengan *username* dan *password* yang sudah dibuat.



Dashboard UNIOT Home About Us Login **Register**

Buat Akun Baru

Username:

Email:

Password:

Konfirmasi Password:

Daftar

[Sudah punya akun? Login di sini](#)

5. Tampilan *dashboard* akan terlihat seperti berikut ini.

Logout

Dashboard Saya

Manajemen Perangkat

Nama Perangkat Baru: Tambahkan Perangkat

Daftar Perangkat Saya

Anda belum mendaftarkan perangkat.

Pilih Perangkat Aktif: Share Link

Anda belum mendaftarkan perangkat...

6. Pada manajemen perangkat, tambahkan perangkat dengan memberi nama seperti contoh “smart home”, lalu klik **Tambahkan Perangkat**. Jika sudah maka akan muncul *Secret Key* (contoh : 5xy3ZY). **Wajib untuk menyimpan secret key** karena jika hilang harus membuat manajemen perangkat yang baru dengan *secret key* yang baru juga. Jika sudah Pilih Perangkat Aktif menjadi “smart home”, maka akan muncul “Perangkat ini belum memiliki widget”.

[Logout](#)

Dashboard Saya

Manajemen Perangkat

Nama Perangkat Baru: [Tambahkan Perangkat](#)

Perangkat Berhasil Didaftar!

Salin Secret Key ini ke ESP32 Anda (Hanya muncul sekali, harap disimpan!):

unot-mssigu4xhjp5be7tlvr

Daftar Perangkat Saya

smart home
[Hapus](#)

Pilih Perangkat Aktif:

smart home

[+ Tambah Variabel](#)

[Share Link](#)

Perangkat ini belum memiliki widget.

7. Klik pada **Tambah Variabel**, lalu isikan dengan *widget* sebagai berikut:

Nama Variabel	Tipe Visualisasi	Tipe Data
Potensiometer	<i>Gauge</i> (Meteran)	<i>Integer</i> (Bulat)
Suhu	<i>Graph</i> (Line Chart)	<i>Float</i> (Desimal)
Kelembapan	<i>Number</i> (Angka)	<i>Float</i> (Desimal)
kondisi-led	<i>Toggle</i> (True/False)	<i>Boolean</i> (True/False)
pwm-led	<i>Slider</i> (PWM)	<i>Integer</i> (Bulan)

8. Klik **Simpan Widget** untuk setiap variabel yang telah ditambahkan, lalu amati apa yang terjadi pada *dashboard*.
9. *Install* dan buka aplikasi WebSocket Tester pada android dari play store.
10. Hubungkan ponsel android ke Wi-Fi UNIOT yang tersedia di laboratorium.

11. Masukkan alamat `ws://ipAddress:port` pada *connection*, lalu klik *Connect*.
12. Isikan `{"action": "auth", "key": "secret-key"}`, dengan secret key yang sudah didapat dari pembuatan *Dashboard*.
13. Isikan `{"var": "Nama Variabel", "val": angka}` pada *message* lalu klik, amati yang terjadi pada *WebSocket Tester* dan *dashboard* UNIOT.
14. Coba klik pada *control* LED pada *dashboard* dan amati pada kotak *input message* (ALL) pada *WebSocket Tester*.

G. Tugas

Jawablah pertanyaan berikut dengan jelas dan tepat.

1. Saat Anda mendaftarkan perangkat baru di *dashboard* UNIOT, sistem menghasilkan sebuah *Secret Key*. Jelaskan fungsi dari *Secret Key* tersebut sesuai dengan pengamatan anda!
2. Jelaskan apa itu *widget* dan sebutkan serta jelaskan macam-macam *widget* yang ada pada *platform* UNIOT!
3. Jelaskan bagaimana metode dan perintah yang dilakukan untuk melakukan *subscribe* ataupun *publish* pada protokol komunikasi *WebSocket*!

H. Dokumentasi

Lampirkan bukti praktikum dengan mengunggah foto *program/dashboard*, foto anda saat praktikum atau *trainer kit*.

Dibuat oleh: Adrian Ramdhany	Dilarang memperbanyak sebanyak atau seluruh isi dokumen tanpa izin tertulis dari Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
------------------------------------	--	--------------------

JOBSHEET 2

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK		
	Semester:	LS 2: Monitoring dan Kontrol <i>Internet of Things</i> berbasis Mikrokontroler ESP32S3	4 x 50 Menit
	Revisi: 00	Tanggal:	

A. Tujuan Praktikum

1. Mahasiswa mampu menunjukkan sikap bertanggung jawab, mandiri dan disiplin dalam menyelesaikan setiap tahapan pekerjaan praktik di bidang mekatronika, khususnya yang berkaitan dengan penerapan sistem *Internet of Things*.
2. Mahasiswa mampu mengonfigurasi dan mempraktikkan komunikasi IoT melalui berbagai *broker* dan *platform* (termasuk UNIoT), baik dalam mode *publisher* maupun *subscriber*, serta mampu mengintegrasikannya dengan aplikasi *mobile* berbasis Android.
3. Merangkai rangkaian elektronik dengan mikrokontroler ESP32S3 sebagai perangkat keras yang melakukan *Subscribe* dan *Publish* pada *platform* UNIoT menggunakan protokol komunikasi *WebSocket*.
4. Melakukan monitoring input (sensor suhu, potensiometer) serta kontrol lampu pada rangkaian *trainer kit*.

B. Capaian Pembelajaran Mata Kuliah

1. Mahasiswa mampu memahami dan mengimplementasikan protokol yang populer digunakan dalam pengembangan sistem berbasis *Internet of Things*.

2. Mahasiswa mampu membuat, merancang dan mengembangkan sebuah sistem sederhana yang menerapkan konsep *Internet of Things*.

C. Dasar Teori

1. *Parsing* JSON

JSON (*JavaScript Object Notation*) adalah format pertukaran data teks yang ringan dan terstruktur berdasarkan pasangan kunci-nilai (*key-value pairs*) (Ismail dkk., 2023). *Parsing* JSON adalah proses komputasi untuk menerjemahkan teks JSON mentah menjadi struktur data asli (*native*) yang dapat dikenali, diakses, dan dimanipulasi oleh bahasa pemrograman.

Proses *parsing* beroperasi melalui dua tahapan utama:

- a. Analisis Leksikal (*Tokenization*): Memecah teks JSON karakter demi karakter menjadi potongan bermakna (token), seperti tanda kurung, koma, teks, dan angka.
- b. Analisis Sintaksis: Memvalidasi susunan token berdasarkan standar aturan JSON, lalu memetakannya menjadi objek atau struktur hierarki di dalam memori program.

Contoh : `{“var”:”suhu”,”val”:25.5}`.

Pada sisi klien yang menggunakan JavaScript, *parsing* JSON berjalan secara natural karena format ini berakar dari bahasa tersebut. Teks mentah diterjemahkan menggunakan fungsi bawaan `JSON.parse()`, yang mengubahnya langsung menjadi objek JavaScript agar datanya dapat di-*render*.

D. Alat dan Bahan

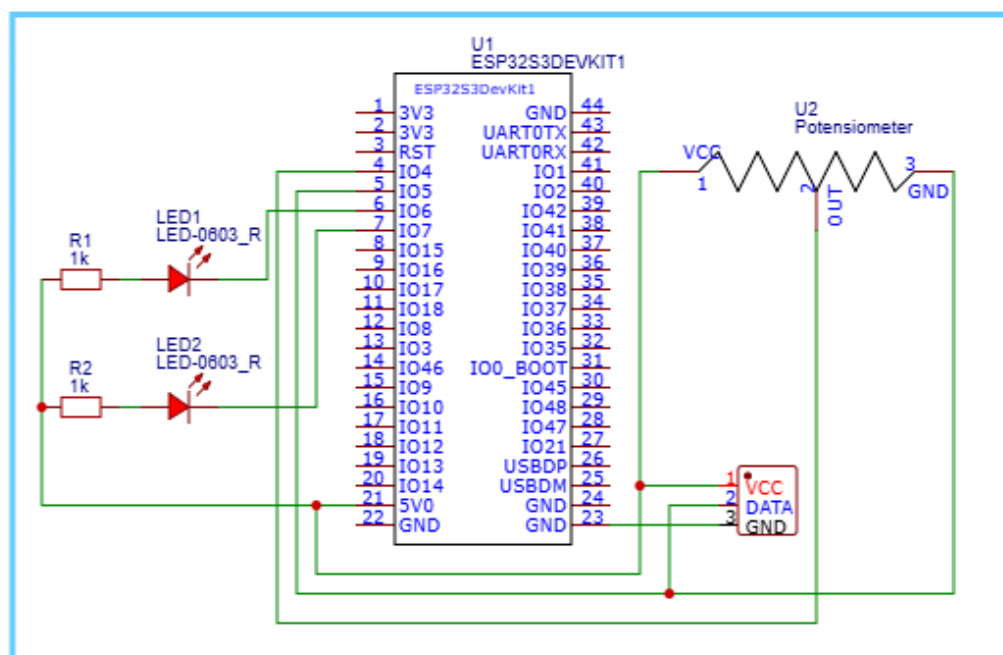
1. Komputer/Laptop dengan aplikasi Arduino IDE.
2. *Trainer Kit* UNIOT.
3. DHT22, Potensiometer, LED, Kabel Jumper.
4. Kabel data USB *type C*.

E. Keselamatan Kerja

- Gunakan pakaian kerja atau jas laboratorium sesuai standar keselamatan saat melakukan kegiatan eksperimen.
- Pastikan area kerja atau meja praktikum selalu dalam keadaan kering, bersih, dan bebas dari paparan cairan.
- Jauhkan benda konduktif yang tidak diperlukan dari area kerja untuk mencegah interferensi elektromagnetik.

F. Langkah Kerja

1. Ambil dan siapkan *Trainer Kit* UNIOT yang sudah tersedia di laboratorium.
2. Mulailah merangkai dengan rangkaian seperti skematik berikut ini.



3. Hubungkan USB ESP32-S3 pada *Trainer Kit* UNIOT ke komputer/laptop.
4. Buka aplikasi **Arduino IDE**.
5. Lakukan instalasi terhadap *library* pendukun berikut ini:
 - **ArduinoJson** (oleh Benoit Blanchon)
 - **WebSockets** (oleh Markus Sattler)

- **DHT Sensor Library** (oleh Adafruit)
 - **Adafruit Unified Sensor**
6. Pastikan *board* dan *port* sudah terpilih dengan benar melalui menu **Tools > Board > ESP32S3 Dev Module** dan cek *port* ESP32-S3 pada **Tools > Port > COM...** (Pilih COM yang terdeteksi).
 7. Pada **Arduino IDE**, pilih menu **File > New** untuk membuat *sketch*/program baru.
 8. Ketik atau salin kode program berikut ini ke dalam Arduino IDE.

```
#include <WiFi.h>
#include <ArduinoJson.h>
#include <WebSocketsClient.h>
#include "DHT.h"
```

```
// =====
```

```
// KONFIGURASI HARDWARE
```

```
// =====
```

```
#define DHTPIN 5
#define DHTTYPE DHT22
#define POT_PIN 4
#define LED_PIN_ONOFF 6
#define LED_PIN_PWM 7
```

```
// =====
```

```
// KONFIGURASI WiFi
```

```
// =====
```

```
const char* ssid = "uniot";
const char* password = "12345678";
```

```
// =====
```

```
// KONFIGURASI UNIoT SERVER (Secret Key)
```

```
// =====
```



```
const char* ws_host = "192.168.0.3"; // IP Server UNIOT
const int ws_port = 3001;           // Port Server
const char* secret_key = "Wnx3UH"; // GANTI dengan secret key
dari dashboard (6 karakter)

// =====
// INISIALISASI GLOBAL
// =====

WebSocketsClient websocket;
DHT dht(DHTPIN, DHTTYPE);

unsigned long lastPotRead = 0;
unsigned long lastDHTRead = 0;
bool isConnected = false;

// Variabel baru untuk melacak nilai potensiometer terakhir yang dikirim
float lastSentPotVal = -100.0; // Diset -100 agar pembacaan pertama
pasti terkirim

// =====
// FUNGSI SUBSCRIBER: Eksekusi Perintah Aktuator
// =====

void eksekusiAktuator(const char* sensorType, const char* value) {

    // Kontrol LED on/off (Pin 6)
    if (strcmp(sensorType, "kondisi-led") == 0) {
        bool state = (strcmp(value, "true") == 0 || strcmp(value, "1") == 0);
        digitalWrite(LED_PIN_ONOFF, state ? HIGH : LOW);
        Serial.printf("EKSEKUSI: LED (Pin 6) diubah menjadi %s\n", state ?
"ON" : "OFF");
    }
}
```

```
// Kontrol LED PWM / Brightness (Pin 7)
else if (strcmp(sensorType, "pwm-led") == 0) {
    int pwmValue = constrain(atoi(value), 0, 255);
    analogWrite(LED_PIN_PWM, pwmValue);
    Serial.printf("EKSEKUSI: PWM (Pin 7) diubah menjadi %d/255\n",
pwmValue);
}

else {
    Serial.printf("Aktuator tidak dikenali: %s\n", sensorType);
}
}

// =====
// CALLBACK: WebSocket Events Handler
// =====

void websocketEvent(WStype_t type, uint8_t* payload, size_t length)
{
    switch (type) {

        case WStype_DISCONNECTED: {
            isConnected = false;
            Serial.println("[WS] Terputus dari server UNIOT");
            Serial.println("[WS] Mencoba menyambung kembali...");
            break;
        }

        case WStype_CONNECTED: {
            isConnected = true;
```

```
Serial.println("[WS] TERHUBUNG ke Websocket UNIoT  
Server!");  
  
Serial.println("[WS] Siap mengirim (Publish) dan menerima  
(Subscribe) data.\n");  
  
break;  
}
```

```
case WType_TEXT: {  
    // Parse JSON perintah dari dashboard  
    StaticJsonDocument<256> doc;  
    DeserializationError error = deserializeJson(doc, payload, length);  
  
    if (error) {  
        Serial.printf("[JSON] Parse error: %s\n", error.c_str());  
        break;  
    }
```

```
    const char* sensorType = doc["var"];
```

```
    const char* value = doc["current_value"];
```

```
    // Fallback cadangan jika format menggunakan "val"  
    if (!value) {  
        value = doc["val"];  
    }
```

```
    if (!sensorType || !value) {  
        Serial.println("[WS] Payload tidak lengkap (kehilangan  
sensor_type atau value)");  
        break;  
    }
```

```
Serial.printf("[WS] Perintah Diterima dari Dashboard: %s = %s\n",
sensorType, value);
    eksekusiAktuator(sensorType, value);
    break;
}

case WStype_PING: {
    Serial.println("[WS] PING diterima dari server");
    break;
}

case WStype_PONG: {
    Serial.println("[WS] PONG dibalas oleh server");
    break;
}

case WStype_ERROR: {
    Serial.printf("[WS] Error: %s\n", payload);
    break;
}

default:
    break;
}
}

// =====
// FUNGSI PUBLISHER: Kirim Data Sensor
// =====
void kirimDataWebSocket(const char* sensor_type, float value) {
```

```
if (!isConnected) return; // Jangan buang memori jika belum konek

StaticJsonDocument<128> doc;
doc["var"] = sensor_type;
doc["val"] = value;

String jsonString;
serializeJson(doc, jsonString);

webSocket.sendTXT(jsonString); // Kirim instan tanpa HTTP POST
Serial.printf("Data Terkirim [%s]: %.2f\n", sensor_type, value);
}

// =====
// SETUP: Inisialisasi Hardware & Koneksi
// =====

void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("\n\n=====
  ===");
  Serial.println("  UNIOT ESP32 Pure WebSocket v2.1");
  Serial.println("  (Secret Key Authentication)");

  Serial.println("=====
  \n");

  // Inisialisasi sensor DHT22
  dht.begin();
```

```
Serial.println("[SENSOR] DHT22 Siap");

// Inisialisasi pin aktuator
pinMode(LED_PIN_ONOFF, OUTPUT);
digitalWrite(LED_PIN_ONOFF, LOW);
pinMode(LED_PIN_PWM, OUTPUT);
analogWrite(LED_PIN_PWM, 0);
Serial.println("[AKTUATOR] LED Siap");

// === Koneksi WiFi ===
Serial.printf("[WiFi] Menghubungkan ke: %s...\n", ssid);
WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);

int wifiTimeout = 0;
while (WiFi.status() != WL_CONNECTED && wifiTimeout < 20) {
    delay(500);
    Serial.print(".");
    wifiTimeout++;
}

if (WiFi.status() == WL_CONNECTED) {
    Serial.printf("\n[WiFi] Terhubung! IP Address: %s\n\n",
WiFi.localIP().toString().c_str());
} else {
    Serial.println("\n[WiFi] Gagal terhubung. Periksa SSID &
Password!");
    return;
}

// === Koneksi WebSocket ke UNIOT Server ===
```

```
Serial.printf("[WS] Membuka terowongan ke: ws://%s:%d\n",  
ws_host, ws_port);
```

```
// Format URL: /?key=0Ly6kU
```

```
String ws_path = String("/") + "?key=" + String(secret_key);
```

```
WebSocket.begin(ws_host, ws_port, ws_path.c_str());
```

```
WebSocket.onEvent(WebSocketEvent);
```

```
WebSocket.setReconnectInterval(5000); // Reconnect otomatis jika  
putus
```

```
Serial.println("[SETUP] Inisialisasi selesai. Menunggu koneksi...\n");  
}
```

```
// =====
```

```
// LOOP: Pembacaan Sensor (Publisher)
```

```
// =====
```

```
void loop() {
```

```
    // WAJIB: Menjaga terowongan WebSocket tetap hidup
```

```
    WebSocket.loop();
```

```
    unsigned long now = millis();
```

```
    // ===== PUBLISH POTENSIOMETER (Hanya jika berubah) =====
```

```
    if (now - lastPotRead > 500) {
```

```
        lastPotRead = now;
```

```
        int rawVal = analogRead(POT_PIN);
```

```
        float potVal = map(rawVal, 0, 4095, 0, 100);
```

```
        // Cek selisih nilai sekarang dengan nilai sebelumnya.
```

```
        if (abs(potVal - lastSentPotVal) >= 1.0) {
```

```

    kirimDataWebSocket("potensiometer", potVal);
    lastSentPotVal = potVal; // Update catatan nilai terakhir yang sukses
    dikirim
  }
}

// ===== PUBLISH DHT22 (Setiap 2000ms / 2 detik) =====
if (now - lastDHTRead > 2000) {
  lastDHTRead = now;

  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();

  if (!isnan(temperature)) kirimDataWebSocket("suhu", temperature);
  else Serial.println("⚠DHT22: Gagal membaca suhu");

  if (!isnan(humidity)) kirimDataWebSocket("kelembapan",
  humidity);
  else Serial.println("DHT22: Gagal membaca kelembapan");
}
}

```

9. Sebelum melakukan *upload*, pastikan Anda sudah mengubah *variable ssid, password, uniot_server* dan *uniot_port* sudah sesuai dengan alamat yang ada pada laboratorium.
10. Klik tombol **Verify** (ikon centang) untuk memastikan tidak ada kesalahan penulisan kode.
11. Jika muncul keterangan *Done compiling*, klik tombol **Upload** (ikon panah ke kanan).
12. Tunggu hingga proses *upload* selesai.
13. Berikut merupakan penjelasan kode untuk masing masing fungsi sebaai berikut:

- a. Fungsi *Subscribe* `void websocketEvent(...)`
Algoritma ini bersiaga menangkap paket data yang masuk dari *dashboard*. Program mendeteksi jenis peristiwa (*event*). Jika paket berisi teks (WStype_TEXT), sistem mengekstrak parameter "var" dan "val" menggunakan pustaka ArduinoJson.
 - b. Fungsi Eksekutor `void eksekusiAktuator(const char* sensorType, const char* value)`
Program memproses hasil ekstraksi JSON dari fungsi *Subscribe*. Sistem membandingkan nilai parameter untuk mengubah status keluaran digital, seperti menghidupkan LED (*digitalWrite*) atau mengatur kecerahan melalui PWM (*analogWrite*).
 - c. Fungsi *Publish* `void kirimDataWebSocket(const char* sensor_type, float value)`
Fungsi ini mengambil nilai dari pembacaan sensor dan dibungkus dalam objek JSON lalu mengirim (*push*) ke *server* UNIOT melalui perintah `websocket.sendTXT`.
 - d. Fungsi *Loop* `websocket.loop()`
Perintah ini wajib berada dalam blok `void loop()`. Perintah ini mengelola pesan masuk dan keluar secara terus menerus (*looping*). Operasi ini menjaga koneksi *WebSocket* terus terbuka agar komunikasi dua arah dipastikan tidak terputus.
14. Buka ***Serial Monitor*** dan set *baud rate* ke 115200. Pastikan muncul tulisan "Perintah Diterima: LED ON / OFF".
 15. Lepaskan koneksi USB jika ingin menggunakan catu daya 220V. Pastikan aman dan tidak ada aliran listrik atau salah rangkaian sebelum menggunakan catu daya 220V.
 16. Buka kembali *browser* dan masuk ke halaman *Dashboard* UNIOT.
 17. Pastikan perangkat aktif adalah perangkat yang sudah dibuat pada *Jobsheet 1*.
 18. Lakukan pengamatan dan perubahan pada masing-masing *widget* mulai dari Potensiometer, sensor suhu dan kelembapan, kondisi-led

dan pwm-led lalu lakukan pengamatan yang terjadi baik pada *dashboard* maupun pada *trainer kit*.

G. Tugas

1. Lakukan pengamatan pada masing-masing *widget* lalu masukkan hasil pengamatan pada table berikut ini.

No	Komponen	Perlakuan	Kondisi
1	Potensiometer	Diputar perlahan hingga mentok ke arah kanan.	Jarum pada <i>widget Gauge</i> di <i>dashboard</i> bergerak naik hingga menunjukkan angka 100 secara <i>real time</i> .
2	Potensiometer	Diputar perlahan hingga mentok ke arah kiri.	Jarum pada <i>widget Gauge</i> di <i>dashboard</i> bergerak naik hingga menunjukkan angka 0 secara <i>real time</i> .
3	DHT22	Api menyala pada jarak 10 cm	
4			
5			
6			
7			
8			
9			

10			
----	--	--	--

2. Berdasarkan praktikum yang telah dilakukan, berikan kesimpulan dari keseluruhan praktikum secara lengkap dan jelas!

H. Dokumentasi

Lampirkan bukti praktikum dengan mengunggah foto *program/dashboard*, foto anda saat praktikum atau *trainer kit*.

Dibuat oleh: Adrian Ramdhany	Dilarang memperbanyak sebanyak atau seluruh isi dokumen tanpa izin tertulis dari Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
------------------------------------	--	-----------------

JOBSHEET 3

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK		
	Semester:	LS 3: Pemrograman <i>Internet of Things</i> menggunakan MIT App Inventor	4 x 50 Menit
	Revisi: 00	Tanggal:	

A. Tujuan Praktikum

1. Mahasiswa mampu menunjukkan sikap bertanggung jawab, mandiri dan disiplin dalam menyelesaikan setiap tahapan pekerjaan praktik di bidang mekatronika, khususnya yang berkaitan dengan penerapan sistem *Internet of Things*.
2. Mahasiswa mampu mengonfigurasi dan mempraktikkan komunikasi IoT melalui berbagai broker dan *platform* (termasuk UNIoT), baik dalam mode *publisher* maupun *subscriber*, serta mampu mengintegrasikannya dengan aplikasi mobile berbasis Android.
3. Merangkai rangkaian elektronik dengan mikrokontroler ESP32S3 sebagai perangkat keras yang melakukan *Subscribe* dan *Publish* pada *platform* UNIoT menggunakan protokol komunikasi *WebSocket*.
4. Melakukan monitoring input (sensor suhu, potensiometer) serta kontrol lampu pada rangkaian *trainer kit*.
5. Mahasiswa/Siswa mampu merancang antarmuka (UI) aplikasi seluler Android secara *native* menggunakan platform *block-building* MIT App Inventor.
6. Mahasiswa/Siswa mampu mengimplementasikan protokol komunikasi *WebSocket* dua arah antara mikrokontroler (ESP32-S3) dan *smartphone* melalui jaringan lokal (Intranet).
7. Mahasiswa/Siswa mampu memecah (*parsing*) paket data berformat JSON untuk menampilkan nilai sensor (suhu, kelembapan, potensiometer) dan merakit JSON untuk mengirim instruksi kontrol aktuator (LED On/Off, PWM).

B. Capaian Pembelajaran Mata Kuliah

1. Mahasiswa mampu memahami dan mengimplementasikan protokol yang populer digunakan dalam pengembangan sistem berbasis *Internet of Things*.
2. Mahasiswa mampu membuat, merancang dan mengembangkan sebuah sistem sederhana yang menerapkan konsep *Internet of Things*.

C. Dasar Teori

1. MIT App Inventor

MIT App Inventor Adalah *platform* pengembangan aplikasi *mobile* berbasis *web* yang bersifat *open-source*. Platform ini pertama kali dirilis pada Maret 2021 dan saat ini dikembangkan oleh *Massachusetts Institute of Technology* (Putri dkk., 2024).

Keunggulan utama dari MIT App Inventor adalah penggunaan metode *Visual Block Programming* yang diadaptasi dari Google Blockly. Melalui metode ini, pengguna tidak perlu mempelajari atau menuliskan sintaks bahasa pemrograman yang rumit secara manual. Proses pembuatan aplikasi dilakukan cukup dengan menyusun, melihat, dan melakukan *drag and drop* blok-blok logika perintah

Platform daring ini sangat ramah bagi pemula namun memiliki kapabilitas yang mumpuni untuk merancang solusi permasalahan di dunia nyata, baik untuk sistem operasi Android maupun iOS. Selain kemudahan dalam merakit antarmuka, MIT App Inventor juga dilengkapi dengan berbagai fitur tingkat lanjut yang terintegrasi, seperti akses pembacaan sensor, *speech recognition*, *text-to-speech*, hingga fitur konektivitas jaringan. Hal ini menjadikannya platform yang sangat ideal untuk dihubungkan dengan perangkat keras dan sistem *Internet of Things* (IoT).

2. Platform UNIOT

Dashboard UNIOT adalah antarmuka berbasis *web* yang bertindak sebagai *Server* IoT. Setiap perangkat fisik yang didaftarkan ke *dashboard* akan mendapatkan *Secret Key*. Kunci ini adalah kode otentikasi unik yang berfungsi mengamankan jalur komunikasi, memastikan data dari ESP32

milik mahasiswa masuk ke akun yang tepat, dan mencegah akses dari pihak luar.

Pada *dashboard* terdapat beberapa *widget* untuk menampilkan data yang diterima dari mikrokontroler ataupun mengontrol *output* pada mikrokontroler. Beberapa *widget* pada *dashboard* adalah sebagai berikut:

1. *Graph (Line Chart)*
 2. *Gauge (Meteran)*
 3. *Number (Angka)*
 4. *Toggle (True / False)*
 5. *Slider (PWM)*
3. *Parsing JSON*

JSON (*JavaScript Object Notation*) adalah format pertukaran data teks yang ringan dan terstruktur berdasarkan pasangan kunci-nilai (*key-value pairs*) (Ismail dkk., 2023). *Parsing JSON* adalah proses komputasi untuk menerjemahkan teks JSON mentah menjadi struktur data asli (*native*) yang dapat dikenali, diakses, dan dimanipulasi oleh bahasa pemrograman.

Proses *parsing* beroperasi melalui dua tahapan utama:

- a. Analisis Leksikal (*Tokenization*): Memecah teks JSON karakter demi karakter menjadi potongan bermakna (token), seperti tanda kurung, koma, teks, dan angka.
- b. Analisis Sintaksis: Memvalidasi susunan token berdasarkan standar aturan JSON, lalu memetakannya menjadi objek atau struktur hierarki di dalam memori program.

Contoh : {“var”:”suhu”,”val”:25.5}.

Pada sisi klien yang menggunakan JavaScript, *parsing JSON* berjalan secara natural karena format ini berakar dari bahasa tersebut. Teks mentah diterjemahkan menggunakan fungsi bawaan `JSON.parse()`, yang mengubahnya langsung menjadi objek JavaScript agar datanya dapat di-*render*.

D. Alat dan Bahan

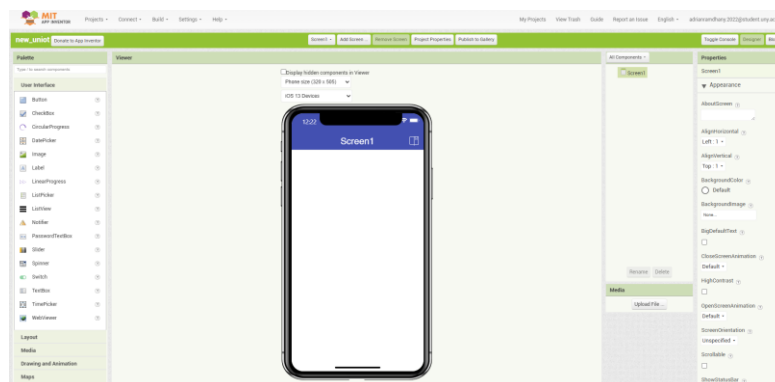
1. Komputer/Laptop dengan aplikasi Arduino IDE.
2. *Trainer Kit* UNIOT.
3. DHT22, Potensiometer, LED, Kabel Jumper.
4. Kabel data USB *type C*.
5. *Smartphone* Android

E. Keselamatan Kerja

1. Komputer/Laptop dengan aplikasi Arduino IDE.
2. *Trainer Kit* UNIOT.
3. DHT22, Potensiometer, LED, Kabel Jumper.
4. Kabel data USB *type C*.

F. Langkah Kerja

1. Buka website ai2.appinventor.mit.edu lalu *login* atau buat akun terlebih dahulu jika belum terdaftar.
2. Buat *new project* dan tampilan akan terlihat seperti berikut ini.



3. Pada tampilan *designer*, masukkan komponen berikut ini.
 - a. *Textbox* dengan judul *ip*.
 - b. *Textbox* dengan judul *port*.
 - c. *Textbox* dengan judul *key*.
 - d. *Button* dengan judul *connect*.
 - e. Label dengan judul status.
 - f. Label dengan judul Label_Suhu.
 - g. Label dengan judul Label_Kelembapan.

- h. *Slider* dengan judul *Gauge_Potensiometer*.
 - i. *Switch* dengan judul *Toggle_LED*.
 - j. *Slider* dengan judul *Pwm_Led*.
 - k. *Webviewer* dengan judul *WebView1*.
 - l. *Connectivity > Web* dengan judul *Web1*.
2. Pada bagian media *upload file* bernama *websocket.html* yang berisikan kode berikut ini.

```
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <style>
    * { margin: 0; padding: 0; }
    body {
      background-color: #222;
      color: #0f0;
      font-family: monospace;
      font-size: 12px;
      padding: 10px;
      overflow-x: hidden;
    }
    #log {
      height: 300px;
      overflow-y: auto;
      border: 1px solid #0f0;
      padding: 10px;
      background: #111;
      margin-top: 10px;
    }
    .error { color: #f00; }
    .success { color: #0f0; }
```



```
.info { color: #0ff; }  
.warn { color: #ff0; }  
b { display: block; margin-top: 10px; margin-bottom: 5px; }  
</style>  
</head>  
<body>  
  <b>Websocket Serial Log (v2.0)</b><br>  
  <div id="log"></div>  
  
  <script>  
    var ws;  
    var logDiv = document.getElementById("log");  
    var messageQueue = [];  
    var connectionStatus = "IDLE";  
    var authenticatedDeviceId = null;  
  
    function printLog(pesan, tipe = 'normal') {  
      var timestamp = new Date().toLocaleTimeString();  
      var className = tipe === 'error' ? 'error' :  
        tipe === 'success' ? 'success' :  
        tipe === 'info' ? 'info' :  
        tipe === 'warn' ? 'warn' : '';  
  
      var html = '<span class="' + className + ">[' + timestamp + ']' +  
        pesan + '</span>';  
      logDiv.innerHTML = html + "<br>" + logDiv.innerHTML;  
  
      if (logDiv.children.length > 100) {  
        logDiv.removeChild(logDiv.lastChild);  
      }  
    }  
  }  
</script>
```

```
function sendToApp(data) {
    try {
        var uniqueKey = "|" + Date.now() + ":" + Math.random();
        var message = data + uniqueKey;
        window.AppInventor.setWebViewString(message);
        console.log("Sent to App: " + message);
    } catch (e) {
        printLog("ERROR sending to App: " + e.message, 'error');
    }
}

function handleServerMessage(event) {
    try {
        var rawData = event.data;
        printLog("<-- RX: " + rawData.substring(0, 100), 'info');

        var json = JSON.parse(rawData);

        if (json.status === 'success' || json.status === 'ack') {
            if (json.status === 'success' && json.device_id !== undefined) {
                authenticatedDeviceId = json.device_id;
                printLog("Auth device_id: " + authenticatedDeviceId,
'success');
            }
            printLog("✓ " + json.message, 'success');
            sendToApp("ACK:" + rawData);
        }
        else if (json.status === 'error') {
            printLog("X ERROR: " + json.message, 'error');
            sendToApp("ERROR:" + rawData);
        }
    }
}
```

```
    }  
    else if (json.type === 'dataUpdate') {  
        printLog("DATA UPDATE: " + json.sensor_type + " = " +  
json.value, 'warn');  
        sendToApp("JSON:" + rawData);  
    }  
    else {  
        printLog("JSON: " + rawData.substring(0, 50), 'info');  
        sendToApp("JSON:" + rawData);  
    }  
  
    } catch (parseError) {  
        printLog("RAW: " + event.data.substring(0, 100), 'warn');  
        sendToApp("RAW:" + event.data);  
    }  
}  
  
setInterval(function() {  
    try {  
        var perintah = window.AppInventor.getWebViewString();  
  
        if (perintah !== "" && perintah !== null) {  
            printLog("--> TX CMD: " + perintah.substring(0, 50), 'warn');  
  
            if (perintah.startsWith("CONNECT:")) {  
                printLog("CMD: CONNECTING...", 'warn');  
                var url = perintah.substring(8);  
  
                if (ws && ws.readyState !== WebSocket.CLOSED) {  
                    ws.close();  
                }  
            }  
        }  
    }  
}
```

```
    printLog("Connecting to: " + url, 'info');
```

```
    ws = new WebSocket(url);
```

```
    connectionStatus = "CONNECTING";
```

```
    ws.onopen = function() {
```

```
        connectionStatus = "CONNECTED";
```

```
        printLog("[OK] WS TERHUBUNG!", 'success');
```

```
        sendToApp("STATUS:CONNECTED");
```

```
    };
```

```
    ws.onclose = function() {
```

```
        connectionStatus = "DISCONNECTED";
```

```
        authenticatedDeviceId = null;
```

```
        printLog("[!] WS TERPUTUS", 'error');
```

```
        sendToApp("STATUS:DISCONNECTED");
```

```
    };
```

```
    ws.onerror = function(error) {
```

```
        printLog("[ERROR] WS Error: " + error.message, 'error');
```

```
        sendToApp("ERROR:CONNECTION_FAILED");
```

```
    };
```

```
    ws.onmessage = handleServerMessage;
```

```
    window.AppInventor.setWebViewString("");
```

```
    }
```

```
    else if (perintah.startsWith("SEND:")) {
```

```
        printLog("CMD: SENDING...", 'warn');
```

```
        var pesan = perintah.substring(5);
```

```
var payloadToSend = pesan;

try {
    var jsonToSend = JSON.parse(pesan);
    var hasVarVal = jsonToSend && jsonToSend.var !==
undefined && jsonToSend.val !== undefined;
    var hasDeviceId = jsonToSend &&
jsonToSend.device_id !== undefined;
    var shouldAutoRoute = hasVarVal && (jsonToSend.type
=== undefined || jsonToSend.type === "command");

    if (shouldAutoRoute && !hasDeviceId &&
authenticatedDeviceId !== null) {
        jsonToSend.type = "command";
        jsonToSend.device_id = authenticatedDeviceId;
        payloadToSend = JSON.stringify(jsonToSend);
        printLog("Auto command route: device_id=" +
authenticatedDeviceId, 'info');
    }
} catch (e) {
}

if (ws && ws.readyState === WebSocket.OPEN) {
    printLog("Sending: " + payloadToSend.substring(0, 50),
'info');
    ws.send(payloadToSend);
    sendToApp("SENT:OK");
} else {
    printLog("WS not connected!", 'error');
    sendToApp("SENT:FAILED");
}
```

```

        window.AppInventor.setWebViewString("");
    }

    else if (perintah.startsWith("STATUS:")) {
        var status = "STATUS:" + (ws ? "WS_READY:" +
ws.readyState + "|" + connectionStatus : "NO_WS");
        sendToApp(status);
        window.AppInventor.setWebViewString("");
    }

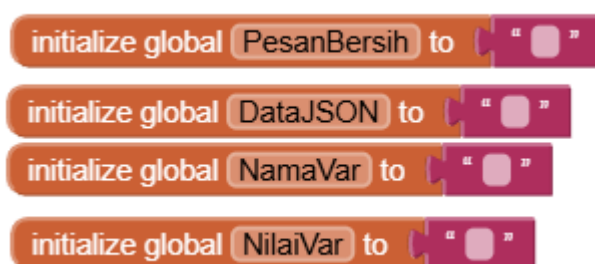
    else if (perintah === "CLEAR") {
        logDiv.innerHTML = "";
        window.AppInventor.setWebViewString("");
    }
}
} catch (e) {
    printLog("ERROR in main loop: " + e.message, 'error');
}
}, 100);

// Initial log
printLog("WebSocket Client Ready!", 'success');
printLog("Commands: CONNECT:<url>, SEND:<json>, STATUS:,
CLEAR", 'info');
</script>
</body>
</html>

```

3. Hilangkan centang *ThumbEnabled* pada *Gauge_Potensiometer* agar parameter tidak bisa diubah.

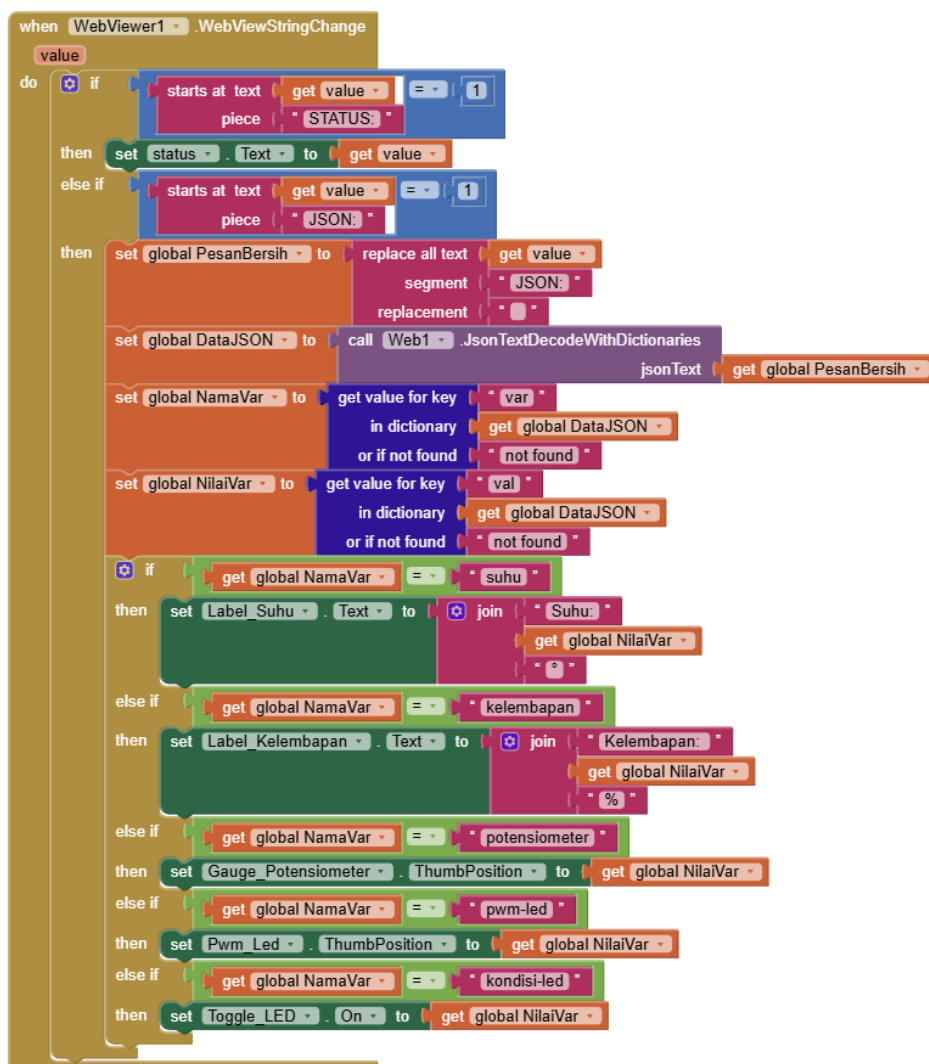
4. Ubah *MaxValue* menjadi 100 dan *MinValue* menjadi 0 dan *Witdh* menjadi *Fill Parent* pada Gauge_Potensiometer dan Pwm_Led.
5. Set *HomeUrl* menjadi <http://localhost/websocket.html> pada *WebView1*.
6. Buka halaman *Blocks* untuk memulai menyusun blok yang akan menjadi otak dari jalannya aplikasi ini.
7. Deklarasikan *global variable* bernama *PesanBersih*, *DataJSON*, *NamaVar* dan *NilaiVar*.



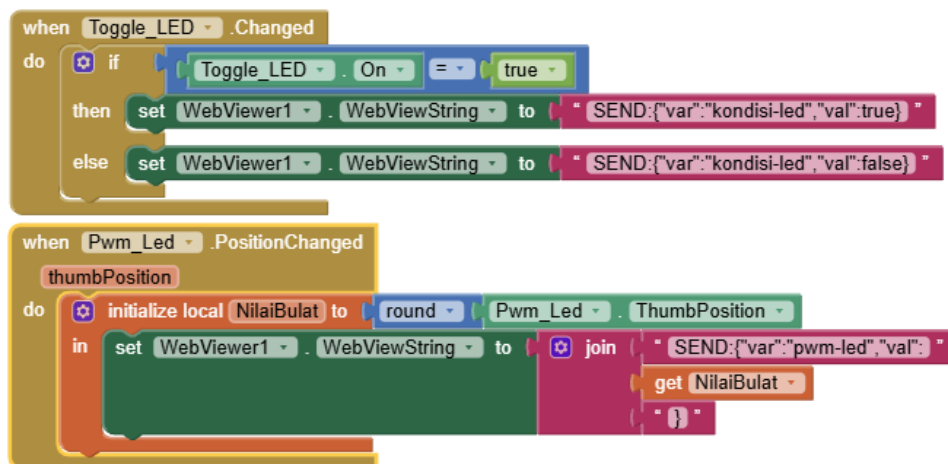
8. Buat logika koneksi dari ip, port, dan *key* yang diinput oleh *user*.



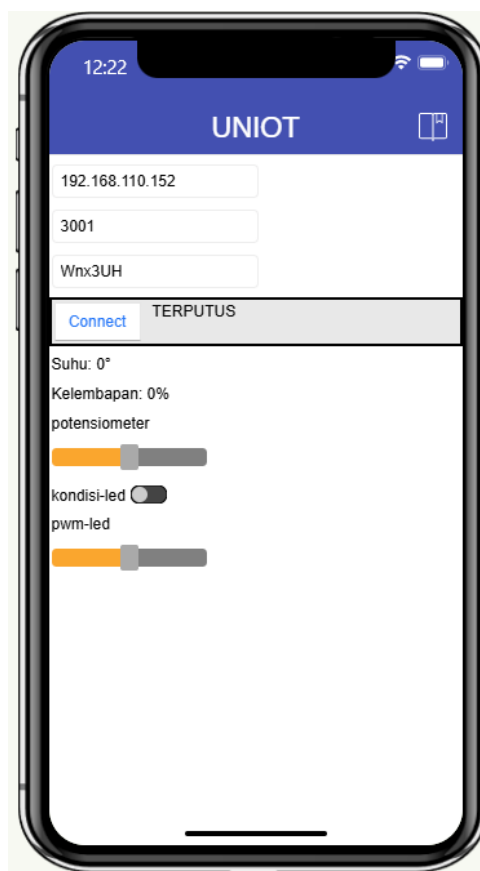
9. Buat blok untuk menerima data JSON dan memisahkannya menjadi beberapa *variable* yaitu untuk suhu, kelembapan dan potensiometer.



10. Untuk memberikan perintah dari aplikasi, buat blok seperti berikut ini untuk mengirim Websocket yang berisi
 {"var":variabel,"val":nilai}



11. *Build* aplikasi menjadi .apk lalu *install* pada *smartphone* Android.
12. Isikan port, ip dan key dengan benar lalu klik Connect. Lalu nyalakan *trainer kit* yang sudah diprogram seperti pada *jobsheet* sebelumnya. Dan coba kontrol menggunakan Aplikasi yang sudah ter-*install* di Android.



G. Tugas

1. Pada arsitektur IoT UNIOT yang telah dibuat pada MIT App Inventor, jelaskan pada bagian mana aplikasi Android bertindak sebagai *Publisher* dan bagian mana yang sebagai *Subscriber*!
2. Sistem komunikasi UNIOT menggunakan metode *multiplexing* di mana data sensor dikirim melalui WebSocket dalam format JSON (Contoh : {"var": "kelembapan", "val": 25.5}). Jelaskan logika pada MIT App Inventor dalam menangani data tersebut agar didapat masing-masing *variable*!

H. Dokumentasi

Lampirkan bukti praktikum dengan mengunggah foto *program/dashboard*, foto anda saat praktikum atau *trainer kit*.

Dibuat oleh: Adrian Ramdhany	Dilarang memperbanyak sebanyak atau seluruh isi dokumen tanpa izin tertulis dari Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
------------------------------------	--	--------------------

PENUTUP

Penyusunan *Jobsheet Platform* UNIOT ini menandai tahap akhir dari perancangan ekosistem media pembelajaran *Internet of Things* di lingkungan laboratorium. Mahasiswa telah menyelesaikan tiga tahapan kegiatan praktikum utama. Kegiatan tersebut mencakup inisialisasi jaringan intranet lokal, integrasi sistem pemantauan sensor fisik, dan pengembangan aplikasi antarmuka seluler berbasis sistem operasi Android. Uji coba pada seluruh tahapan praktikum ini membuktikan bahwa protokol *WebSocket* dan manajemen basis data MySQL mampu mengeksekusi transmisi aliran data secara presisi pada arsitektur jaringan tertutup.

Penggunaan *Trainer Kit* berspesifikasi mikrokontroler ESP32-S3 berhasil meningkatkan keterampilan praktis mahasiswa dalam merancang dan memecahkan masalah (*troubleshooting*) pada perangkat keras IoT secara mandiri. Penulis berharap modul panduan ini memberikan kontribusi empiris dalam memperbarui kualitas kurikulum vokasi pada bidang keilmuan mekatronika. Penulis merekomendasikan peneliti selanjutnya untuk memperluas kapabilitas platform UNIOT dengan mengintegrasikan algoritma pembelajaran mesin (*machine learning*) guna meningkatkan efisiensi analisis data lingkungan.

Yogyakarta, 23 April 2026

Adrian Ramdhany
NIM 22518241019